# SPACE COLONY

## Android Application

Object-Oriented Programming – Project Documentation

Spring 2026

| | |
|---|---|
| **Course** | Object-Oriented Programming |
| **Project** | Space Colony – Android App |
| **Deadline** | April 20, 2026 at 23:59 (Finnish time) |
| **Language** | Java (Android Studio) |

# 1. Project Description

Space Colony is an Android application built in Java using Android Studio. The app simulates managing a space station crew where the player recruits crew members with different specializations, trains them in a simulator, and sends them on cooperative turn-based missions against system-generated threats.

The application follows the Object-Oriented Programming paradigm throughout, making use of inheritance, encapsulation, and polymorphism across the crew member class hierarchy.

## 1.1 Core Concept

- Recruit crew members with 5 different specializations: Pilot, Engineer, Medic, Scientist, Soldier
- Move crew between three locations: Quarters, Simulator, and Mission Control
- Train crew in the Simulator to gain experience points (XP), increasing their effective skill
- Send two crew members on cooperative missions against system-generated threats
- Missions are turn-based: crew act, threat retaliates, until one side is defeated
- Surviving crew earn XP; defeated crew are removed from the roster
- Returning crew to Quarters fully restores their energy

## 1.2 Application Flow

Home Screen shows a summary of how many crew are in each location. From there the player navigates using the bottom navigation bar to Quarters, Simulator, Mission Control, or Statistics. Crew are recruited from the Quarters screen and can be moved to the Simulator for training or to Mission Control for missions. In Mission Control the player selects a mission type and two crew members, then fights the threat round-by-round choosing Attack, Defend, or Special each turn.

## 2. UML Class Diagram

The diagram below describes the model and manager layer of the application. UI classes (Fragments, Adapters, MainActivity) are excluded as per the documentation requirements.

```
<<abstract>>
CrewMember
──────────────────────────────────────
+ id: int
+ name: String
+ baseSkill: int
+ resilience: int
+ experiencePoints: int
+ currentEnergy: int
+ maxEnergy: int
+ location: Location
+ statistics: Statistics
──────────────────────────────────────
+ act(threat, missionType): int  <<abstract>>
+ getSpecialization(): String  <<abstract>>
+ defend(damage: int): int
+ getEffectiveSkill(): int
+ addExperience(xp: int)
+ recoverEnergy()
+ isAlive(): boolean
```

Subclasses (all extend CrewMember):

| Pilot | Engineer | Medic | Scientist | Soldier | Bonus Mission |
|---|---|---|---|---|---|
| SKL:5 RES:4 HP:20 Bonus: NAVIGATION | SKL:6 RES:3 HP:19 Bonus: REPAIR | SKL:7 RES:2 HP:18 Bonus: MEDICAL | SKL:8 RES:1 HP:17 Bonus: RESEARCH | SKL:9 RES:0 HP:16 Bonus: COMBAT | +3 skill on matching type |

Other classes:

| Class | Key Members |
|---|---|
| **Threat** | + name, skill, resilience, energy \| + act(): int \| + defend(dmg): int \| + isAlive(): boolean |
| **Storage** | - HashMap<Integer, CrewMember> crewMap \| + addCrew() \| + removeCrew() \| + getCrewByLocation(Location) |
| **MissionEngine** | - List<CrewMember> squad \| - Threat currentThreat \| + startMission(MissionType) \| + executeRound(actions, type): String |
| **DataManager** | + saveData() — saves crew to JSON file \| + loadData() — loads crew from JSON on app start |
| **Statistics** | + missions: int \| + wins: int \| + trainingSessions: int \| incrementMissions/Wins/Training() |
| **Location (enum)** | QUARTERS \| SIMULATOR \| MISSION_CONTROL |
| **MissionType (enum)** | COMBAT \| REPAIR \| NAVIGATION \| MEDICAL \| RESEARCH |

# 3. Implemented Features

## 3.1 Mandatory Features

- Object-Oriented paradigm: abstract CrewMember base class, 5 concrete subclasses, encapsulation, inheritance, polymorphism
- All code, comments, and documentation written in English
- Android app built in Java with Android Studio, runs on Android devices
- Crew Management: recruit Pilot / Engineer / Medic / Scientist / Soldier; newly recruited crew placed in Quarters
- Move crew between Quarters, Simulator, and Mission Control
- Training System: crew gain +1 XP per training session in Simulator; effective skill = baseSkill + XP
- Cooperative Mission System: select 2 crew, generate scaled threat, turn-based combat loop
- Threat scaling: skill = 4 + missionCount, resilience = 1 + missionCount/2, energy = 15 + missionCount*3
- Mission outcome: surviving crew gain XP, defeated crew are permanently removed
- Crew Recovery: returning to Quarters fully restores energy, XP is retained
- Data Structures: HashMap<Integer, CrewMember> in Storage, ArrayList for location-filtered lists

## 3.2 Bonus Features Implemented

The following bonus features are implemented and should be graded:

| Feature | Points | Status | Where |
|---|---|---|---|
| RecyclerView for crew lists | +1 | YES | `All screens` |
| Crew Images / Color badges | +1 | YES | `CrewAdapter, item_crew.xml` |
| Tactical Combat (Attack/Defend/Special) | +2 | YES | `MissionEngine, MissionControlFragment` |
| Statistics tracking per crew member | +1 | YES | `Statistics.java, StatsFragment` |
| Randomness in missions (skill + random 0-2) | +1 | YES | `MissionEngine.executeRound()` |
| Specialization Bonuses per mission type | +2 | YES | `Pilot/Engineer/Medic/Scientist/Soldier.act()` |
| Data Storage & Loading (Gson JSON) | +2 | YES | `DataManager.java` |
| Fragments used meaningfully | +2 | YES | `All 6 fragments` |

**Total potential score: 13 (mandatory) + 12 (bonus) = 25 points**

# 4. Application Use-Flow

The application uses a single Activity (MainActivity) with a bottom navigation bar connecting five Fragments.

| Screen | What the user can do |
|---|---|
| **Home** | Overview: see how many crew are in each location. Navigate to any section. |
| **Quarters** | See crew at home. Recruit new crew member (name + specialization). Move selected crew to Simulator or Mission Control. |
| **Simulator** | Train selected crew (+1 XP each). Send crew back to Quarters (energy restored). |
| **Mission Control** | Select mission type. Select 2 crew members. Launch mission. Each round: choose Attack, Defend, or Special. View live threat HP bar and crew HP bars. After mission ends, return to selection or go Home. Send crew back to Quarters without fighting. |
| **Statistics** | View per-crew stats: missions completed, wins, training sessions. |

# 5. Installation Instructions

## Option A – Run from Android Studio

1. Clone or download the repository from GitHub
2. Open Android Studio and select Open > project folder
3. Wait for Gradle sync to complete
4. Connect an Android device or start an emulator (API 24+)
5. Press Run (green play button)

## Option B – Install APK directly

- The compiled debug APK is located at: app/build/outputs/apk/debug/app-debug.apk
- Enable 'Install from unknown sources' on your Android device
- Transfer the APK and install it

## Requirements

- Android Studio Hedgehog or later
- Android SDK API 24 (Android 7.0) minimum, API 34 target
- Java 8+ (source compatibility)
- Gradle 8.11
- Gson library (included in dependencies)

## 6. Team Composition & Division of Work

| Name | Student ID | Responsibilities |
|---|---|---|
| Md Maksudur Rahman Mahi | 002295583 | Full project (solo submission) |

### Work Breakdown

- Model layer (CrewMember, subclasses, Threat, Statistics, enums) — core OOP design
- Manager layer (Storage, MissionEngine, DataManager) — game logic and persistence
- UI layer (all 6 Fragments, layouts, navigation graph, bottom nav) — Android UI
- RecyclerView adapter (CrewAdapter) with checkbox selection and specialization colors
- Mission system: tactical combat with Attack/Defend/Special, threat scaling, XP rewards
- Bonus features: Gson save/load, statistics, specialization mission bonuses, randomness
- Testing, bug fixing, and documentation

## 7. Tools Used

| Tool | Purpose |
|---|---|
| **Android Studio** | Primary IDE for Android development |
| **Java 8** | Programming language |
| **Gradle 8.11** | Build system |
| **Gson (Google)** | JSON serialization for save/load feature |
| **Android Navigation Component** | Fragment navigation and back stack |
| **RecyclerView** | Efficient scrollable crew lists |
| **Android ProgressBar** | Health bar visualization for crew and threats |
| **Material Design / BottomNavigationView** | App navigation UI |
| **GitHub** | Version control and project submission |

## 8. GitHub & Video Links

| Resource | Link |
|---|---|
| **GitHub Repository** | https://github.com/md-maksudur-rahman-mahi/Space_colony_project/tree/main |
| **Demo Video** | Part 1(codes): https://youtu.be/kroAKmvj7U4<br>part 2 (app): https://youtu.be/5vgHuhmWHgo |

## 9. AI Usage Declaration

**AI was used in this project.**

The following describes where and how AI assistance was used:

| Area | Tool Used | How it was used |
|---|---|---|
| Code review & bug fixing | Claude (Anthropic) | Identifying bugs in MissionControlFragment.java, fixing a missing method body, and replacing incompatible var keyword |
| UI improvements | Qwen | Suggestions for adding crew health bars during missions and return-to-quarters button in Mission Control |
| Project documentation | Chatgpt | Generating the structure and content of this documentation file |

All core logic, OOP design, class structure, Android UI implementation, and game mechanics were written by the student. AI was used as an assistant tool for reviewing and improving existing code, not for generating the primary implementation from scratch.

*— End of Documentation —*